

ISyE6644 Project Report

Group 208, Alvaro Larreátegui

2025-07-20

1. Introduction

This project consists in **using Monte Carlo simulation to develop a news vendor problem application.**

Monte Carlo simulation (MCS) was primarily developed by S. Ulam in the 1940s by realizing “that electronic computers made it practical to apply statistical methods to functions without known solutions”. A. Law defines MCS as “a scheme employing random numbers, that is, $U(0,1)$ random variates, which is used for solving certain stochastic or deterministic problems” [1]. Robert and Casella state: “[the] central concept of Monte Carlo methods, that is, taking advantage of the availability of computer-generated random variables to approximate univariate and multi-dimensional integrals” [2]. With current computational power, MCS can draw thousands of simulated samples that can be used to compute many statistics about each sample. Then, thanks to the Law of Large Numbers, it is possible to compute probabilities and confidence intervals for the expected values with precision that increases as the number of simulations increases.

The **News Vendor Problem** (NVP) is a classic problem in inventory management, that appeared in 1888 in the context of banking [3]. A newsboy must determine the number of newspapers to order for the next day, considering that if he orders too many he will incur in overage costs (papers left unsold), and if he orders too few, he will face underage costs (lost sales). The decision must be made daily, prior to the realization of the demand for the day. This problem has been usually approached as an optimization problem: finding the optimal quantity to order such that it maximizes the expected profits or equivalently minimizes the expected costs. The classic version of the problem, involving a single product, a single period (SPP) has an exact analytical solution called the Critical Fractile solution [4, 5]. Regarding applications, the NVP model is suitable for cases in which we are not able to store inventory from one period to the next, plus some sort of underage and overage costs apply. Categories of products such as style goods and perishable items fall into this class. For example, a retailer might want to know the optimal amount of fresh fruit, vegetables or flowers to order such as minimizing the costs associated with lost sales and waste.

2. Problem Statement and Model Specification

The specific problem of **Optimizing the Daily Production of a Bakery** is approached from the perspective of the NVP, i.e. find the production amount Q that maximizes the expected profit. The approach here is to consider a **Compound Demand**, which is a general demand model that models the number of customers, e.g. N as a random variable and the demand of a specific item, e.g. X as another random variable (see for example [4]). The demand of a single item k for a single day can be expressed as:

$$D_k = \sum_{i=1}^N X_{ik}$$

Contrary to the Single Period Problem (SPP), the model above has no easy analytical solution in the general case [4]. An alternative is to find Q by simulation, which is the approach here. By generating many realizations of the random variables that compose the demand, we will compute the daily profits for a business cycle, say a quarter or year, then average them. By doing this for many replications, the Law of Large Numbers says the average profit is normally distributed. This allows us to have a point estimator of the expected profit as well as a confidence interval.

Five items are included in the model here, thus the aggregated daily demand is:

$$D = \sum_{k=1}^5 D_k$$

On the other hand, the daily profit P_k of item k is a function of the daily demand D_k and the daily quantity Q_k produced. If $D_k \leq Q_k$, there is inventory left in excess that represents a loss. It is assumed here that bakery products can be sold only on the day they are produced, ie no salvage value is considered here. If $D_k > Q_k$, all the daily inventory is sold in full, but there will be unsatisfied demand, which does not have a direct material effect on daily profit. But since it may have an impact in the long term, one way of include this effect in the model is via a penalty factor.

The daily profit for item k is easily expressed as:

$$D_k \leq Q_k : P_k = m_k D_k - c_k (Q_k - D_k) \quad (1)$$

$$D_k > Q_k : P_k = m_k Q_k - p_k (D_k - Q_k) \quad (2)$$

with:

- m_k , unit margin for item k ,
- c_k , unit cost for item k ,
- p_k , shortage penalty term for item k

The first term in above equations, mD or mQ is the sales profit. The second term is a loss: - in (1), $c(Q - D)$, corresponds to an excess of inventory at the end of the day. - in (2), $p(D - Q)$, corresponds to an excess of demand (ie product shortage).

Equations (1) and (2) can be combined in the general case as:

$$P_k = m_k \cdot \min(D_k, Q_k) - c_k (Q_k - D_k)^+ - p_k (D_k - Q_k)^+ \quad (3)$$

where:

- $(Q_k - D_k)^+ = \max(Q_k - D_k, 0) \doteq EI_k$, is the excess inventory, if any (0 otherwise).
- $(D_k - Q_k)^+ = \max(D_k - Q_k, 0) \doteq ED_k$, is the excess demand, if any (0 otherwise).

The daily profits for each product are arranged into a row vector of profits for the items considered. Let $P_d = [P_1, P_2, P_3, P_4, P_5]$ be the vector of daily profits for a given day d in a simulation. We can write (3) in matrix form to consider one simulation run of D days as follows:

$$\mathbf{P} = \mathbf{m} * \min(\mathbf{D}, \mathbf{Q}) - \mathbf{c} * \mathbf{EI} - \mathbf{p} * \mathbf{ED} \quad (4)$$

Equation (4) above only matrices of same dimensions, where column k corresponds to item k , and row d corresponds to day d of a simulation run. The $*$ symbol corresponds to the Hadamard product (element wise product) of matrices and the $\min$ is also taken element wise. For example, for one run of 365 days, the matrices will have dimension $(365, 5)$. Matrices $\mathbf{m}$, $\mathbf{Q}$, $\mathbf{c}$ and $\mathbf{p}$ correspond to the margin, quantities, cost and penalty information, that varies in the columns but is replicated in the rows. This matrix formulation makes coding much easier and efficient.

The following economic data is considered for our model with the five top-selling items from the dataset **French Bakery Daily Sales** from Kaggle [6], that is used to model the demand:

	Unit Price	Unit Cost	Unit Margin
TRADITIONAL BAGUETTE	1.30 €	0.90 €	0.40 €
CROISSANT	1.20 €	0.80 €	0.40 €
PAIN AU CHOCOLAT	1.30 €	0.85 €	0.45 €
COUPE	0.15 €	0.10 €	0.05 €
BANETTE	1.15 €	0.85 €	0.30 €

Unit prices are extracted from the dataset, but unit costs were estimated at around 65% of price based on the following information: <https://www.lefigaro.fr/conjoncture/combien-cela-coute-t-il-de-produire-une-baguette-de-pain-20220120> (<https://www.lefigaro.fr/conjoncture/combien-cela-coute-t-il-de-produire-une-baguette-de-pain-20220120>).

Finally, the penalty cost is considered as a percentage of the margin; that for simplicity is assumed to be the same for all products. The objective is to materially incorporate in the model some sort of penalty factor for when the demand is not met. In general, demand that is not satisfied is hard to record (see [7]).

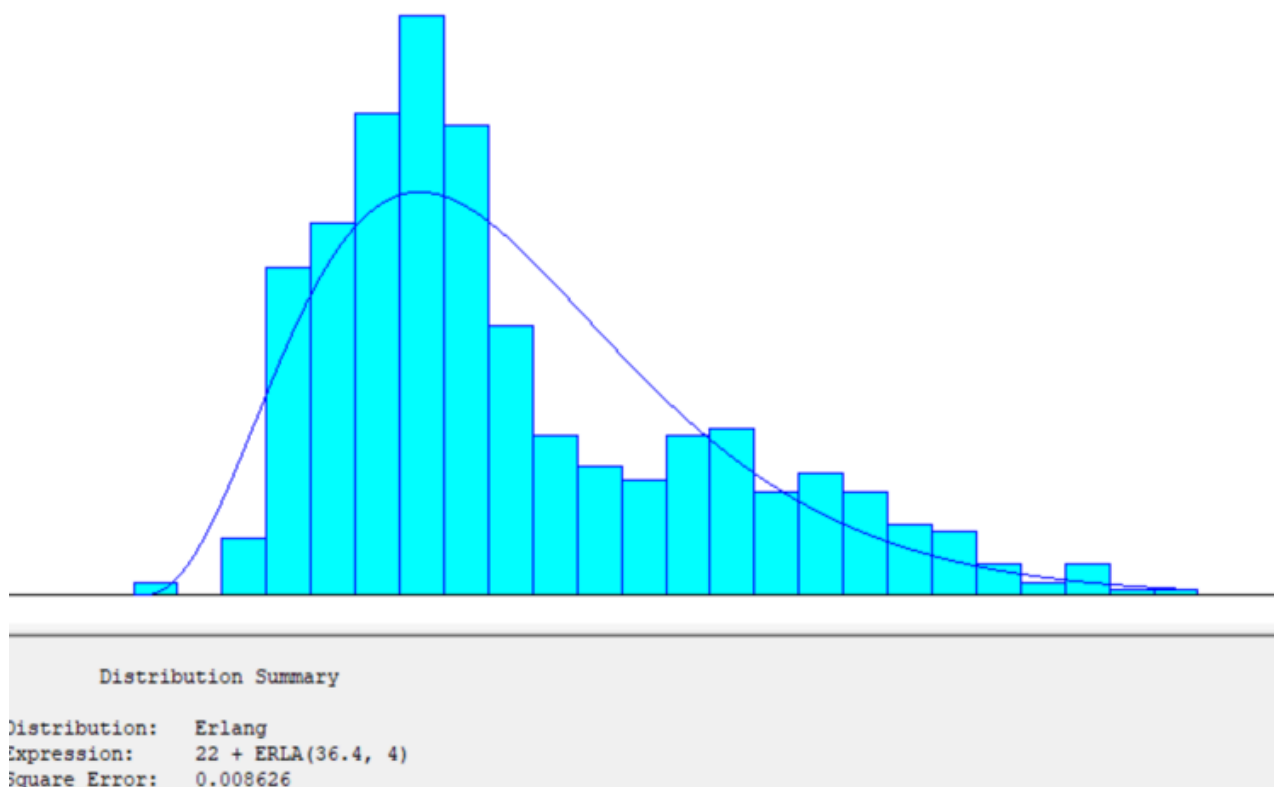
3. Input Analysis

Real demand data is hard to obtained, especially for traditional walk-in stores as in this case. Real sales data are used here as a proxy of real demand. The dataset **French Bakery Daily Sales** from Kaggle [6] is used to model the demand. It consists of transactional sales data with detail on date, time and items purchased, from Jan-01-2021 to Sep-30-2022. This is a screenshot of the first records:

date	time	ticket_number	article	Quantity	unit_price
1/2/2021	8:38	150040	BAGUETTE	1	0,90 â‚¬
1/2/2021	8:38	150040	PAIN AU CHOCOLAT	3	1,20 â‚¬
1/2/2021	9:14	150041	PAIN AU CHOCOLAT	2	1,20 â‚¬
1/2/2021	9:14	150041	PAIN	1	1,15 â‚¬
1/2/2021	9:25	150042	TRADITIONAL BAGUETTE	5	1,20 â‚¬
1/2/2021	9:25	150043	BAGUETTE	2	0,90 â‚¬
1/2/2021	9:25	150043	CROISSANT	3	1,10 â‚¬
1/2/2021	9:27	150044	BANETTE	1	1,05 â‚¬
1/2/2021	9:32	150045	TRADITIONAL BAGUETTE	3	1,20 â‚¬
1/2/2021	9:32	150045	CROISSANT	6	1,10 â‚¬
1/2/2021	9:37	150046	PAIN AU CHOCOLAT	6	1,20 â‚¬
1/2/2021	9:37	150046	CROISSANT	6	1,10 â‚¬
1/2/2021	9:37	150046	TRADITIONAL BAGUETTE	6	1,20 â‚¬
1/2/2021	9:39	150048	CROISSANT	3	1,10 â‚¬

It is assumed that each customer is uniquely identified by the ticket number. For example, the first customer in the screenshot above buy 1 baguette and 3 pain au chocolat. The demand model approach is twofold: the first step is to aggregate the unique ticket numbers by day, to obtain the daily arrivals. The second step is to model the purchases. This will allow to model the demand as a **Compound Demand** as expressed previously.

The data aggregation into daily arrivals was performed in excel, and the resulting vector of random arrivals was fed into Arena's Input Analyzer to fit a distribution. The best fit was obtained with the "FitAll" option that resulted in an shifted $Erlang_4(\lambda = 1/36.4)$ distribution. This was obtained for the full 2021-2022 period, but excluding Sundays (because of demand peaks). See below a picture of the AI output (see full fit in Annex 1).



With this fit, the arrivals can be modeled as the sum of 4 exponential random variables with parameter $\lambda = 1/36.4$, that is shifted by +22.

The second phase to find the purchase probabilities by item was also performed in excel. This time, the quantity purchased was aggregated by day (excluding Sundays) and by customer, to get a frequency table that was translated into probabilities (see below). This was done per item, thus assuming here that the amounts purchased are independent. No particular distribution was fit this time, just used as empirical distribution. The table was readjusted to a quantity of 9 per customer, for simplicity.

Item Quantities Purchase Probabilities					
Quantity	Baguette	Croissant	PainChocolat	Coupe	Banette
0	0.33225	0.88684	0.89586	0.80778	0.85096
1	0.35901	0.03439	0.03512	0.16232	0.09824
2	0.19982	0.03626	0.03481	0.02291	0.03542
3	0.06226	0.01573	0.01379	0.00478	0.00863
4	0.02763	0.01265	0.00988	0.00136	0.00395
5	0.00998	0.00557	0.00480	0.00043	0.00166
6	0.00425	0.00422	0.00287	0.00012	0.00060
7	0.00177	0.00112	0.00088	0.00005	0.00013
8	0.00113	0.00166	0.00091	0.00009	0.00031
9	0.00021	0.00016	0.00017	0.00002	0.00002
10	0.00137	0.00090	0.00066	0.00005	0.00006
11	0.00001	0.00007	0.00002	0.00000	0.00000
12	0.00015	0.00019	0.00007	0.00001	0.00001
13	0.00003	0.00004	0.00001	0.00001	0.00000
14	0.00000	0.00001	0.00002	0.00001	0.00000
15	0.00006	0.00006	0.00004	0.00002	0.00000
16	0.00000	0.00001	0.00000	0.00000	0.00000
17	0.00000	0.00003	0.00000	0.00000	0.00000
18	0.00000	0.00001	0.00003	0.00001	0.00000
19	0.00000	0.00000	0.00000	0.00000	0.00000
20	0.00006	0.00006	0.00004	0.00003	0.00002
21	0.00000	0.00000	0.00000	0.00001	0.00000
22	0.00000	0.00000	0.00000	0.00000	0.00000
23	0.00000	0.00000	0.00000	0.00000	0.00000
24	0.00000	0.00000	0.00000	0.00000	0.00000
25	0.00001	0.00001	0.00001	0.00000	0.00000

4. Model Implementation

The model is implemented in R and as a Shiny interactive application. In this section, the general flow and functions used in the app code are explained in detail,, as well as depicted with a few examples.

The only global parameter in the code is $\lambda = 1/36.4$, the rate of arrivals:

```
lambda <- 1/36.4
```

The app uses different helper functions included in the **helpers.R** file. They are all explained in detail in this section, along with simple examples.

4.1. Function to generate daily customer arrivals

This function generates customers arrivals as a RV with an Erlang($n=4$, $\lambda=1/36.4$), that is obtained as the sum of 4 easily generated $\text{expo}(1/36.4)$

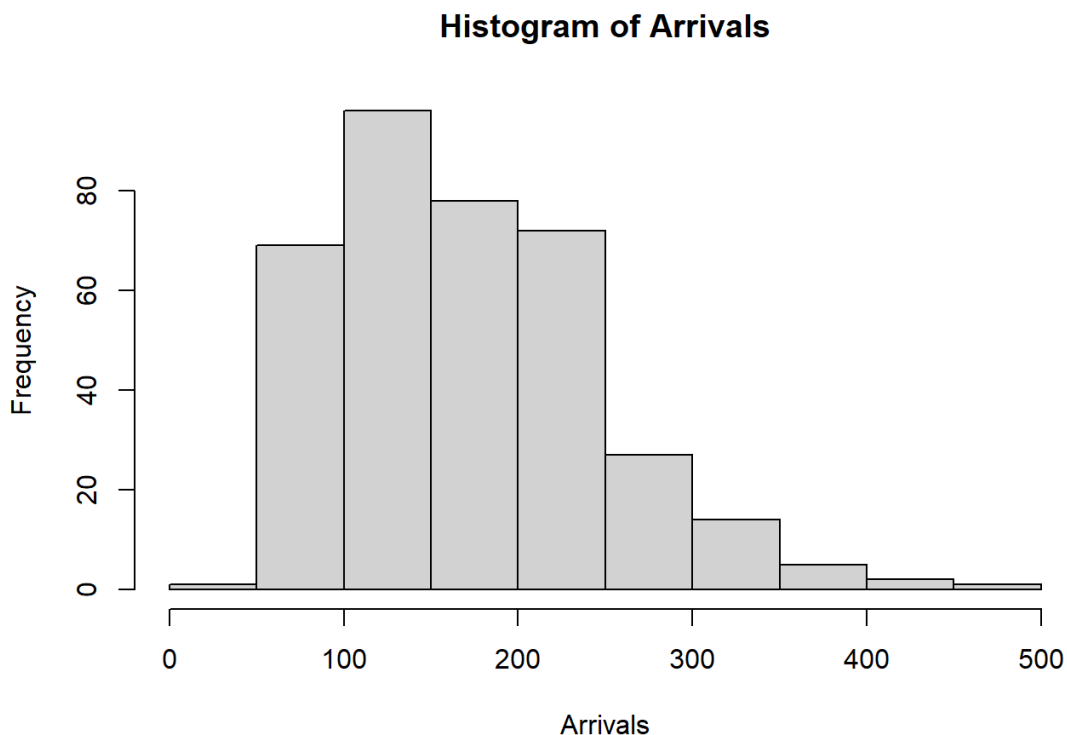
```
arrivals <- function(lambda, NDAYS){
  ### Generate Daily Arrivals
  # In: lambda, rate of Erlang distribution
  #      NDAYS, number of days to simulate
  # Out: A, (NDAYS) vector of # of Erlang daily arrivals

  # generate 4 exponential RVs as row vectors
  u <- matrix(data = runif(NDAYS*4), nrow = 4)
  x <- -(1/lambda) * log(u, base = exp(1))

  # generate daily arrivals (customers) as sum of the 4 expos
  A <- 22 + ceiling(1 + colSums(x))
  return(A)
}
```

For example, this is a realization for one run:

```
Arrivals <- arrivals(lambda = 1/36.4, 365)
hist(Arrivals)
```



4.2. Functions to Generate the compound demand

The following function returns the empirical p.m.f. of each product in matrix form (items are in columns). This way we have this data self-contained in the code.

```
get_item_prob <-function(){
  # helper function to return the empirical distribution of the demand of each product
  P <- matrix(data = c( 0.333773913, 0.887931549, 0.896517799, 0.807781367, 0.850962972,
    0.359006211, 0.034394689, 0.035120148, 0.162315528, 0.098244986,
    0.199821118, 0.036262994, 0.034812076, 0.022906832, 0.035418282,
    0.062260870, 0.015731521, 0.013793652, 0.004780124, 0.008626001,
    0.027627329, 0.012650806, 0.009878163, 0.001361491, 0.003945302,
    0.009977640, 0.005565162, 0.004799952, 0.000427329, 0.001659611,
    0.004253416, 0.004223561, 0.002872021, 0.000119255, 0.000596267,
    0.001768944, 0.001122970, 0.000884463, 0.000049689, 0.000129191,
    0.001132919, 0.001659611, 0.000914277, 0.000089441, 0.000308072,
    0.000377640, 0.000457137, 0.000407449, 0.000168943, 0.000109316 ),
    byrow = TRUE,
    ncol = 5)
  colnames(P) <- c('Baguette', 'Croissant', 'PainChoco', 'Coupe', 'Banette')
  rownames(P) <- c('P(X=0)', 'P(X=1)', 'P(X=2)', 'P(X=3)', 'P(X=4)', 'P(X=5)', 'P(X=6)', 'P(X=7)', 'P(X=8)', 'P(X=9)')
  return(P)
}
```

This is a snapshot of the matrix:

```
P <- get_item_prob()
head(round(P, 3),5)
```

```
##          Baguette Croissant PainChoco Coupe Banette
## P(X=0)    0.334    0.888    0.897 0.808    0.851
## P(X=1)    0.359    0.034    0.035 0.162    0.098
## P(X=2)    0.200    0.036    0.035 0.023    0.035
## P(X=3)    0.062    0.016    0.014 0.005    0.009
## P(X=4)    0.028    0.013    0.010 0.001    0.004
```

The function below assigns random purchases to each customer, by generating a sample from the empirical distribution above. Then they are aggregated to have daily purchases per product, per day in matrix form.

```
item_demand <- function(A){
  ### Generate daily demand as an aggregate of the item demand across all the customers
  # In: A, (NDAYS) vector of # of Erlang daily arrivals
  # Out: DD (NDAYS,nprod) matrix of aggregated demand per day, per product

  PP <- get_item_prob()
  DD <- matrix(data = NA, nrow = length(A), ncol = dim(PP)[2])

  for (i in 1:length(A)) {
    # for each product generate a vector of daily aggregate demand
    for (j in 1:dim(PP)[2]){
      # for a single day generate a sample of product demand for each customer and aggregate
      temp_x <- sample(x = 0:9, size= A[i], prob = PP[,j], replace = TRUE)
      DD[i,j] <- sum(temp_x)
    }
  }
  colnames(DD) <- c('Baguette', 'Croissant', 'PainChoco', 'Coupe', 'Banette')
  return(DD)
}
```

For example, for six simulated business days, a possible aggregated demand could be:

```
set.seed(1)
(AAo <- arrivals(lambda = 1/36.4, 4))
```

```
## [1] 132 103 262 107
```

```
(DDo <- item_demand(AAo))
```

```
##          Baguette Croissant PainChoco Coupe Banette
## [1,]    141         21         55   28     29
## [2,]    112         38         30   29     12
## [3,]    302         78         82   48     62
## [4,]    142         15         19   29     20
```

4.3. Functions to Compute the daily profit

With the aggregated daily demand we can compute the daily profit as a function of the production levels per product (stored in a vector Q) and the economic data. We start by retrieving the cost and margin information:

```
ec_params <- function(){
  # helper function to retrieve the economic params
  EC <- data.frame(mrg = c(0.4, 0.7, 0.6, 0.1, 0.3),
                  cst = c(0.9, 0.5, 0.7, 0.05, 0.85))
  return(EC)
}
```

The following helper functions allow to compute the excess inventory (EI) and excess demand (ED) matrices:

```

row2matrix <- function(x, n){
  # creates a matrix with vector x as n replicated rows
  M <- matrix(x, nrow = 1)
  M <- M[rep(1,n),]
  return(M)
}
excess_inventory <- function(DD, Q){
  # returns the excess inventory if there is, 0 otherwise
  Qm <- row2matrix(Q, nrow(DD))
  EI <- pmax(Qm - DD, 0)
  return(EI)
}
excess_demand <- function(DD, Q){
  # returns the excess demand if there is, 0 otherwise
  Qm <- row2matrix(Q, nrow(DD))
  ED <- -pmin(Qm - DD, 0)
  return(ED)
}

```

Here is how these work with our previous daily demand matrix:

```
cat('\nDemand\n')
```

```
##
## Demand
```

```
(DDo)
```

```
##      Baguette Croissant PainChoco Coupe Banette
## [1,]      141        21        55    28      29
## [2,]      112        38        30    29      12
## [3,]      302        78        82    48      62
## [4,]      142        15        19    29      20
```

```
cat('\n(Arbitrary) Production policy\n')
```

```
##
## (Arbitrary) Production policy
```

```
Q <- c(150, 50, 50, 100, 100)
Qm <- row2matrix(Q,4)
colnames(Qm) <- c('Baguette', 'Croissant', 'PainChoco', 'Coupe', 'Banette')
(Qm)
```

```
##      Baguette Croissant PainChoco Coupe Banette
## [1,]      150        50        50   100   100
## [2,]      150        50        50   100   100
## [3,]      150        50        50   100   100
## [4,]      150        50        50   100   100
```

```
cat('\nExcess inventory\n')
```

```
##
## Excess inventory
```

```
(EI <- excess_inventory(DDo, Q))
```

```
##      Baguette Croissant PainChoco Coupe Banette
## [1,]      9      29        0    72    71
## [2,]     38     12       20    71    88
## [3,]      0      0        0    52    38
## [4,]      8     35       31    71    80
```

```
cat('\nUnsatisfied demand\n')
```

```
##
## Unsatisfied demand
```

```
(ED <- excess_demand(DDo, Q));
```

```
##      Baguette Croissant PainChoco Coupe Banette
## [1,]      0      0        5     0     0
## [2,]      0      0        0     0     0
## [3,]    152     28       32     0     0
## [4,]      0      0        0     0     0
```

Finally, we have all we need for implementing the computation of the model in equation (4), the matrix of daily profits:

```
profit_mat <- function(DD, Q, pct_pen){
  # Compute a matrix of profits by day and by product
  # In: DD, matrix of aggregated demand per day, per product
  #     Q, vector of production policy
  #     pct_pen, shortage penalty as % of margin
  # Out: TP: (nrpod) vector of total profits for one simulation
  EI <- excess_inventory(DD, Q)
  ED <- excess_demand(DD, Q)
  EC <- ec_params()
  MRG <- row2matrix(EC$mrg, nrow(DD))
  CST <- row2matrix(EC$cst, nrow(DD))
  PEN <- row2matrix(pct_pen*EC$mrg/100, nrow(DD))
  PM <- MRG * pmin(Q,DD) - CST * EI - PEN * ED
  return(PM)
}
```

For our example, this is how the Profit Matrix looks like, along with the profit per item and total profit for one simulation run:

```
(PM <- profit_mat(DDo, Q, 20))
```

```
##      Baguette Croissant PainChoco Coupe Banette
## [1,]    48.30     0.20    32.40 -0.80  -51.65
## [2,]   -14.20    20.60     4.00 -0.65  -71.20
## [3,]    7.84    31.08    45.36  2.20  -13.70
## [4,]    32.80    -7.00   -10.30 -0.65  -62.00
```

Notice that our arbitrary policy yields almost all positive daily profits for *Baguette*, but all negative ones for *Banette*.

4.3. Functions to Optimize the profit in one simulation run

For a fixed realization of the demand, we can vary the quantities Q_k to see which ones maximize the profit: this is done in this section. A key assumption is that the demands for each product are independent. Thus we can maximize the profit of each product separately. For each product, the profit will increase until as more units are sold, until a maximum is reached. Then, the excess inventory cost will start eroding the profit.

An example is depicted below:

```

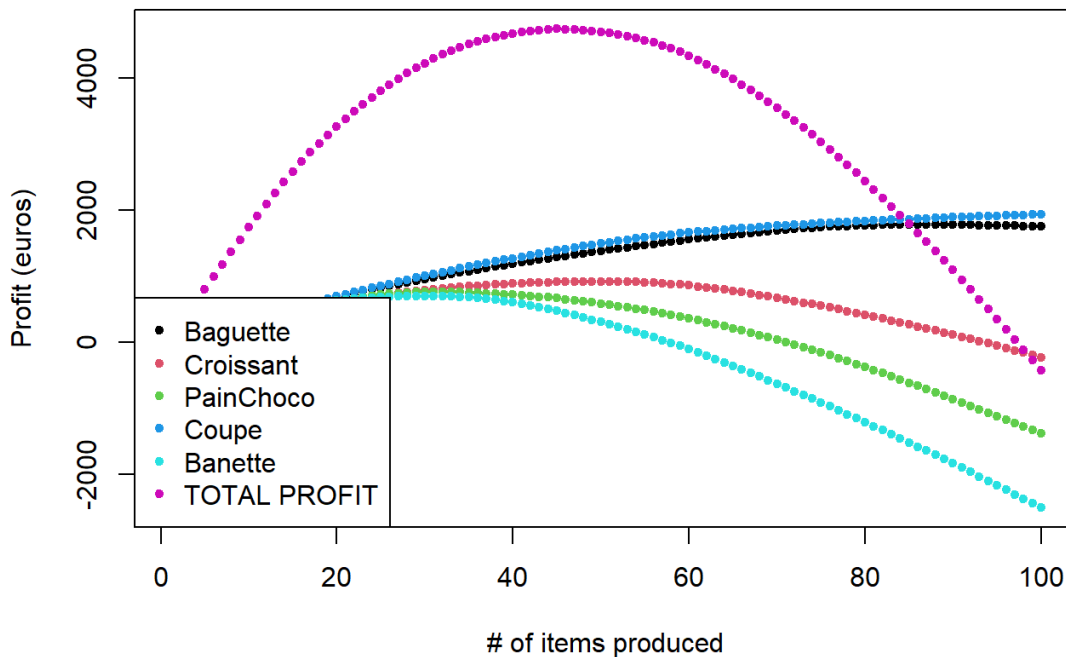
plot_optimization <- function(NDAYS, PCT_PEN, IMAX){
  DD <- item_demand(arrivals(lambda = 1/36.4, NDAYS))
  profit <- matrix(data = 0, nrow = IMAX, ncol = 5)
  for(i in 1:IMAX ){
    profit[i,1] <- sum(profit_mat(DD, c(i, 0, 0, 0, 0), PCT_PEN/100))
    profit[i,2] <- sum(profit_mat(DD, c(0, i, 0, 0, 0), PCT_PEN/100))
    profit[i,3] <- sum(profit_mat(DD, c(0, 0, i, 0, 0), PCT_PEN/100))
    profit[i,4] <- sum(profit_mat(DD, c(0, 0, 0, i, 0), PCT_PEN/100))
    profit[i,5] <- sum(profit_mat(DD, c(0, 0, 0, 0, i), PCT_PEN/100))
  }
  Tprofit <- cbind(profit, rowSums(profit) )
  matplot(Tprofit, type = c("p"), pch = 20, col = 1:6, xlab = '# of items produced', ylab = 'Profit (euros)')
  legend("bottomleft", legend = c('Baguette', 'Croissant', 'PainChoco', 'Coupe', 'Banette', 'TOTAL PROFIT'), col=1:6, pch=20)
}

```

```

plot_optimization(NDAYS = 90, PCT_PEN = 20, IMAX = 100)

```



Notice how *Banette* decreases faster after reaching the maximum profit. This is likely due to its high cost and low margin combined to a low demand for it. Something similar happens to *Pain au chocolat* and *Croissant*, though their higher demand seems to counteract this effect. *Coupe* has low demand, but very low cost, so having excess inventory is less serious. Finally, the *Baguette* has a high unit cost but also the highest demand, so there is low risk of having it overstocked.

Ok, so from above we realize that the optimization algorithm is simple: to compute the profit incrementally for each item, starting at $Q_k = 0$, until a decrease in profit is reached.

```

optimize_profit <- function(DD, pct_pen, toplot=FALSE){
  ### Find quantities of each product that maximize Total Profit
  # In: DD, matrix of aggregated demand per day, per product
  #     ECON: data frame of economic parameters
  # Out: Qx, (5) vector of optimal quantities
  Qmin <- apply(DD, MARGIN = 2, FUN = min)
  TPo <- colSums(profit_mat(DD, Qmin, pct_pen))
  Qx <- c(0, 0, 0, 0, 0)
  TP <- TPo
  Q <- Qmin
  TPlot <- c()
  for (j in 1:5){
    TP <- colSums(profit_mat(DD, Q, pct_pen))
    TPlot <- append(TPlot, sum(TP))
    Qx[j] <- Q[j]
    while (TP[j] - TPo[j] >= 0){
      Qx[j] <- Q[j]
      Q[j] <- Q[j] + 1
      TPo <- TP
      TP <- colSums(profit_mat(DD, Q, pct_pen))
      TPlot <- append(TPlot, sum(TP))
    }
  }
  if(toplot){
    return(TPlot)
  } else {
    return(Qx)
  }
}

```

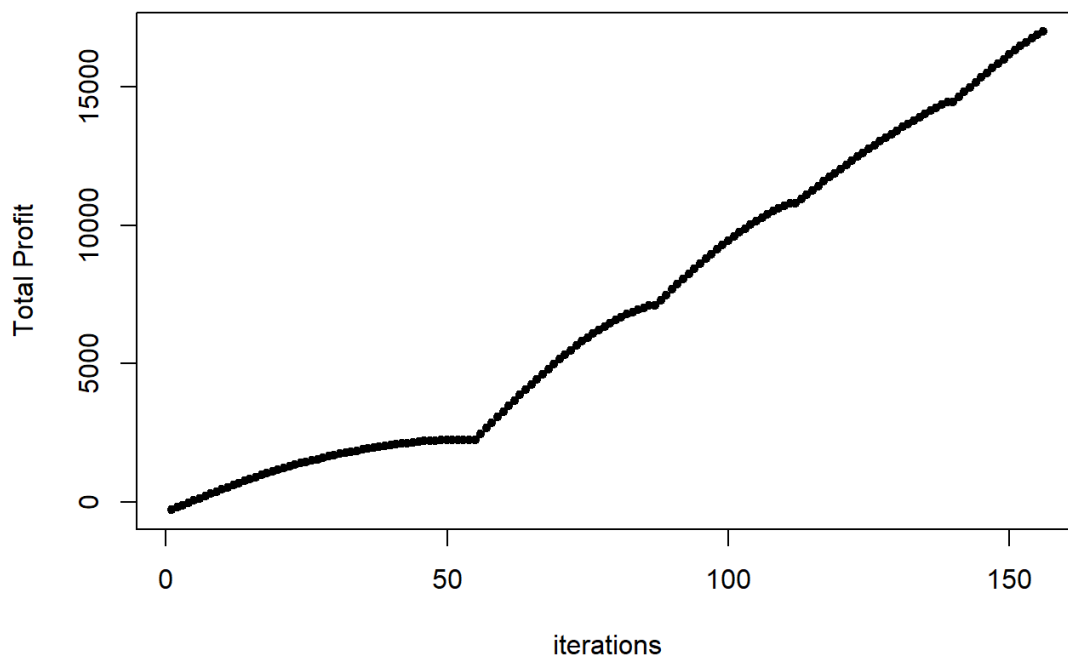
Below an example for one run:

```

DDo <- item_demand(Arrivals)
plot(optimize_profit(DDo, pct_pen=20, toplot=TRUE),
     main='optimization of total profit across all items',
     xlab = 'iterations', ylab='Total Profit', pch=20)

```

optimization of total profit across all items



The function returns a vector with the Optimal Production Policy, that we can plug into *profit_mat* and then compute the mean daily profit:

```
(Qo <- optimize_profit(DDo, pct_pen=20))
```

```
## [1] 105 35 27 33 17
```

```
(mean(profit_mat(DDo, Qo, pct_pen=20)))
```

```
## [1] 9.102378
```

Now, we need to be able to gather the optimal policies for every replication, to hopefully get a good estimator of the mean daily profit. For large numbers of replications, by the Law of Large Numbers, it have low variance and low bias. We do that with the following function:

```
run_sims <- function(lambda, NREP, NDAYS, PEN, SEED=1){
  set.seed(SEED)
  BEST_Q <- matrix(data = NA, nrow = NREP, ncol = 5)
  BEST_TP <- matrix(data = NA, nrow = NREP, ncol = 1)
  EC <- ec_params()
  for (i in (1:NREP)){
    A <- arrivals(lambda, NDAYS)
    DD <- item_demand(A)
    BEST_Q[i,] <- optimize_profit(DD, PEN)
    BEST_TP[i] <- mean(profit_mat(DD, BEST_Q[i,], PEN))
  }
  BEST_Q <- cbind(BEST_Q, BEST_TP)
  colnames(BEST_Q) <- c('Baguette', 'Croissant', 'PainChoco', 'Coupe', 'Banette', 'AVG_PROFIT')
  BEST_Q <- as_tibble(BEST_Q)
  return(BEST_Q)
}
```

Let's see an example for 500 runs of 365 days, with a 5% shortage penalty applied:

```
BEST_Q <- run_sims(lambda=1/36.4, NREP=500, NDAYS=365, PEN=5)
```

The output tibble contains the optimal production policie for every day as well as the average daily profit:

```
head(BEST_Q)
```

```
## # A tibble: 6 × 6
##   Baguette Croissant PainChoco Coupe Banette AVG_PROFIT
##   <dbl>     <dbl>    <dbl> <dbl>   <dbl>    <dbl>
## 1     101       33      22   29     17     11.0
## 2     103       30      21   28     17     10.6
## 3      94       28      19   29     15     10.0
## 4     104       32      23   30     18     11.0
## 5      96       29      21   29     15     10.1
## 6     103       30      20   30     17     10.7
```

A 95% Confidence Interval shows how packed together

```
cat('\n 95% CI for the Average Daily Profit:\n')
```

```
##
## 95% CI for the Average Daily Profit:
```

```
(MeanCI(BEST_Q$AVG_PROFIT))
```

```
##      mean   lwr.ci   upr.ci
## 10.76658 10.73361 10.79956
```

```
cat('\n 95% CI for the Production Policies:\n')
```

```
##
## 95% CI for the Production Policies:
```

```
(ceiling(apply(BEST_Q[,1:5], 2, MeanCI)))
```

```
##      Baguette Croissant PainChoco Coupe Banette
## mean      100       31       23    30    17
## lwr.ci     99       31       22    30    17
## upr.ci    100       32       23    30    17
```

For this particular case (using 10% shortage penalty) the french baker can be confident that 95% of the times the true daily profit will be between 10.77 and 10.80 when producing 100 Baguettes, 30 Croissants, 23 Pains au Chocolat, 30 Coupes and 17 Banettes.

5. App implementation

The functions described in the previous section were implemented in the file called **helpers.R** that is a source for the Shiny app that is attached to the project. The code for the Shiny app is in the file **app.R**. Instructions on how to run it are provided in the README file.

6. Conclusion

This project was a good opportunity to implement important simulation concepts to solve in a relatively easy manner the otherwise difficult problem of optimizing the production of a bakery with a compound demand of product. The example described at the end of section 4 shows how with large number of replications with large number of days, a robust solution can be obtained. For the few examples that I was able to do in the app, the optimal policies were similar to that of that example. I hope the users will find it interesting to use.

7. References

- [1] A. Law. Simulation Modeling and Analysis. 5th edition, 2015, p714
- [2] C.P. Robert, G. Casell. Monte Carlo Statistical Methods. 2nd edition, 2004, p79
- [3] F.Y. Edgeworth (1888), "The Mathematical Theory of Banking", Journal of the Royal Statistical Society. 51 (1): 113–127
- [4] Guillermo Gallego (18 Jan 2005). "IEOR 4000 Production Management Lecture 7"(PDF). Columbia University. Retrieved 22 June 2025.
- [5] M. Khouja (1999), "The single-period (news-vendor) problem: literature review and suggestions for future research", Omega The international Journal of Management Sciences. 27: 537-553
- [6] <https://www.kaggle.com/datasets/matthieugimbert/french-bakery-daily-sales>
(<https://www.kaggle.com/datasets/matthieugimbert/french-bakery-daily-sales>)
- [7] A.L.Sachs. Retail Analytics, 2015, p35